CHAPTER 1

INTRODUCTION

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

This project focuses on implementing an **image encryption system using the Advanced Encryption Standard (AES)**, combining both **Verilog** and **Python** to create a complete hardware–software solution. The goal is to ensure secure storage and transmission of digital images by converting them into an encrypted format that cannot be interpreted without the correct key.

In this work, **Verilog** is used to design and simulate the AES encryption module at the hardware level, demonstrating how image data can be processed efficiently using digital logic. **Python** is utilized for image preprocessing, data conversion, and post processing tasks, enabling seamless integration between raw image inputs and the hardware encryption module. Together, these components form a robust pipeline that reads an image, encrypts it using AES, and outputs a secure encrypted image.

This project highlights the practical application of cryptographic algorithms in hardware design and showcases the advantages of combining FPGA-based processing with software-based data handling.

1.2 Objectives

The primary objectives of this project focus on designing a secure, efficient, and hardware optimized system for image encryption and decryption using the Advanced Encryption Standard (AES) algorithm

1.2.1 To design and implement an AES-based image encryption and decryption system using Verilog HDL

This objective aims to build a complete hardware model of the AES algorithm in Verilog. The system includes modules for key expansion, encryption rounds, and decryption rounds. Verilog HDL is chosen because it allows the design to be implemented on hardware platforms such as FPGAs, enabling high-speed and real-time encryption of images. Implementing AES in hardware also provides better control over timing, data flow, and parallel operations compared to software.

1.2.2 To improve encryption performance by utilizing hardware-level parallel processing

Traditional software encryption executes AES operations sequentially, which can be time-consuming for large images. In contrast, hardware implementations allow multiple operations—such as SubBytes, ShiftRows, MixColumns, and AddRoundKey—to run simultaneously or in pipelined stages. This significantly increases throughput and reduces latency. The objective is to exploit this hardware parallelism to accelerate the overall encryption and decryption process, making the system suitable for high-speed applications like surveillance, real-time video systems, and embedded security.

1.2.3 To ensure secure protection of digital image data during transmission and storage

Security is the central requirement of this project. The system must ensure that when an image is encrypted, it becomes completely unreadable to unauthorized users. Even if intercepted, the cipher image must not reveal any meaningful information about the original image. AES is chosen because of its proven security strength and resistance to cryptographic attacks. This objective ensures that confidential images—such as medical scans, biometric data, or military images—remain protected both during transmission across networks and while stored in memory or cloud systems.

1.2.4 To generate an encrypted image that visually appears random and cannot be interpreted without the secret key

A good encryption system should produce an output image that shows no visual similarity to the original. Once AES encryption is applied, all pixel values are randomized, eliminating patterns, edges, or identifiable structures. Achieving this ensures that even if an attacker views the encrypted image, they cannot guess the content. This objective focuses on validating the quality of encryption through randomness, histogram uniformity, and correlation analysis of pixel values.

1.2.5 To accurately reconstruct the original image after decryption using the same key

Since AES is a symmetric key algorithm, the same 128-bit key used for encryption must also correctly decrypt the image. The decrypted image should be identical to the original, with no loss of visual quality or pixel distortion. This objective involves ensuring that the decryption module is fully functional and correctly reverses all AES transformations—Inv Sub Bytes, Inv Shift Rows, Inv Mix Columns, and Add Round Key—to retrieve the original data. Successful reconstruction indicates the correctness of the hardware design.

1.2.6 To analyze performance in terms of speed, security strength, and resource utilization

A complete evaluation of the hardware design is essential for understanding its efficiency. This objective involves:

Speed Analysis: Measuring encryption and decryption time, clock cycles, throughput, and latency.
Security Strength: Checking randomness of cipher image, pixel correlation, entropy values, and resistance to brute-force attacks.
Resource Utilization: Examining FPGA resource usage such as LUTs, flip-flops, registers, memory blocks, and power consumption.

By analyzing these parameters, the project determines whether the proposed AES architecture is optimized and suitable for real-world applications.

CHAPTER 2

LITERATURE SURVEY

CHAPTER 2

LITERATURE SURVEY

H. Pahuja Kumar and B. Singh [2016]

This paper proposes a high-speed AES (Advanced Encryption Standard) design using techniques derived from ancient Vedic mathematics. The authors aim to optimize the cryptographic algorithm by minimizing computational delays, particularly in the multiplication operation. Vedic mathematics is used to improve arithmetic operations in AES's transformation stages, such as MixColumns. The design is tested using Verilog and simulated on FPGA, demonstrating enhanced performance over traditional methods. This novel integration of ancient math with modern cryptography presents a fresh angle in AES hardware design.

R. Borhan, R. M. Fuad, and Tengku Aziz [2012]

This paper presents the design of the AES encryption algorithm using Verilog HDL. The focus is on developing a functional model that can be synthesized on FPGA for real-time This work focuses on the hardware implementation of the AES algorithm using FPGA. The authors aim to demonstrate the practical feasibility of AES in real-time hardware systems. Their implementation is written in VHDL and verified through simulation and synthesis. The results show that the algorithm can be efficiently realized on FPGA platforms with high throughput and low latency. The paper underscores the benefits of hardware implementation in securing embedded systems, emphasizing power and area optimization.

M. Gunasekaran, K. Rahul, and S. Yachareni [2021]

This study implements a 256-bit key AES algorithm on a Virtex-7 FPGA platform. The key contributions include optimization of the Key Schedule and SubBytes block, which are crucial for performance and security. The work emphasizes reducing latency and improving throughput by modifying these core AES functions. The hardware is modeled using VHDL and tested using simulation tools, showing improvements in speed and efficiency compared to earlier implementations. The paper demonstrates that high-bit AES can be efficiently handled by modern FPGA devices with optimization.

V. H. Soumya, M. B. Neelagar, and K. V. Kumaraswamy [2018]

This paper presents the design of the AES encryption algorithm using Verilog HDL. The focus is on developing a functional model that can be synthesized on FPGA for real-time secure communication. The authors explain each AES round and its implementation in Verilog. Simulation results validate the correctness of the design, and the study claims a balance between performance and resource utilization. The work contributes to educational and practical aspects of learning AES design through hardware description languages.

J. Sunil, S. H. S, S. B. K, and S. Santhameena [2020]

This comparative study explores the AES algorithm's implementation on both hardware (FPGA) and software platforms. The authors analyze and compare parameters such as execution speed, resource usage, and security. The paper concludes that hardware implementation outperforms software in speed and security aspects, making it suitable for embedded and IoT applications. The work provides a comprehensive insight into trade-offs between hardware and software implementations of AES.

P. Katkade and G. M. Phade (2016)

In their paper titled "Application of AES Algorithm for Data Security in Serial Communication" presented at the 2016 International Conference on Inventive Computation Technologies (ICICT), explore the implementation of the Advanced Encryption Standard (AES) algorithm in the context of securing data transmission over serial communication channels. The study addresses the growing concern over the security of serial communication, especially in embedded systems and industrial applications, where sensitive data is often transmitted over relatively unprotected channels. The authors propose using AES, a symmetric key encryption standard, due to its balance of strong security and computational efficiency.

A. Kumar and S. Tejani [2019]

This paper deals specifically with the architecture of the S-Box in AES encryption. It discusses different techniques to optimize the S-Box, including look-up tables and logic gate based approaches. The aim is to enhance the performance of this non-linear substitution stage, which is key to the cryptographic strength of AES. The authors review several architectural variants and their effect on power, area, and speed. This focused study aids designers in selecting or developing efficient S-Box implementations for AES in various applications.

M. R. Rao and R. K. Sharma [2017]

The paper demonstrates how the combined design improves area efficiency, memory usage, and power consumption compared to separate S-Box and Inv S-Box implementations. The architecture is implemented and tested on FPGA, and results show reduced LUT usage and overall hardware footprint without compromising the performance required for real-time encryption and decryption. This work highlights the importance of optimized substitution box design for achieving lightweight and efficient AES hardware systems.

CHAPTER 3

METHODOLOGY

CHAPTER 3

METHODOLOGY

3.1 Introduction

The methodology adopted in this project focuses on developing a complete hardware software co-design framework for implementing AES based image encryption using Verilog and Python. The approach integrates digital hardware design principles with high-level software processing to ensure secure, efficient, and reliable image encryption suitable for real-time and embedded applications.

The overall workflow begins with preprocessing the image in Python, where pixel data is extracted, formatted, and converted into 128-bit data blocks compatible with the AES hardware module. This ensures that the image information aligns with the block-based structure of the encryption algorithm. Python also facilitates initial verification through software-based AES encryption, enabling a reference model for validating hardware functionality.

The core component of the system the AES encryption engine is implemented using Verilog at the register transfer level (RTL). The hardware module includes key expansion, Sub Bytes, Shift Rows, Mix Columns, and Add Round Key operations, designed to operate sequentially or with pipelining depending on synthesis constraints. Functional simulation, waveform analysis, and synthesis are performed to verify correctness, evaluate timing performance, and estimate hardware resource utilization.

Once the hardware module is validated, the encrypted data produced by the Verilog AES core is transferred back to the Python environment. Python reconstructs the encrypted image by mapping the encrypted blocks back into pixel format. This not only completes the encryption pipeline but also allows visual inspection and software-level verification of the hardware-generated ciphertext.

This combined methodology ensures that the strengths of both hardware and software are utilized—Verilog providing high-speed, parallelizable encryption capabilities, and Python offering flexibility for data management, image handling, and verification. The step-by-step flow enables a clear understanding of system behavior, supports modular testing, and provides a scalable foundation for future enhancements such as FPGA implementation, real-time encryption, or integration with embedded security systems.

3.2 Design Flow Overview

The design flow of the proposed AES-based image encryption system follows a structured hardware–software co-design approach. The process begins with **image preprocessing in Python**, where the input image is converted into a stream of 128-bit blocks suitable for AES encryption. These blocks, along with the encryption key, are then supplied to the **AES hardware module designed in Verilog**, which performs key expansion and executes the AES rounds to generate encrypted data.

Functional simulation and verification are carried out using Verilog tools to ensure the correctness of each AES stage. After successful hardware encryption, the resulting ciphertext blocks are transferred back to Python, where they are **reconstructed into an encrypted image** for visualization and validation. This integrated flow ensures secure, high-speed AES processing in hardware while utilizing Python for flexible image handling, analysis, and verification.

3.3 WORKING DIAGRAM OF AES

Image Encryption

• **Image encryption** is the process of transforming a readable image (plain image) into an unreadable and scrambled form called an **encrypted image** or **cipher image**. This is achieved using a mathematical algorithm and an encryption key. The key determines how the pixel values of the original image are converted into the encrypted form, ensuring that the image cannot be understood or viewed without proper authorization.

Image Decryption

• **Image decryption** is the reverse process of image encryption. It converts the encrypted or cipher image back into the original plain image using the correct decryption key. Only a user with the correct key can successfully decrypt the encrypted image and recover the original visual information.

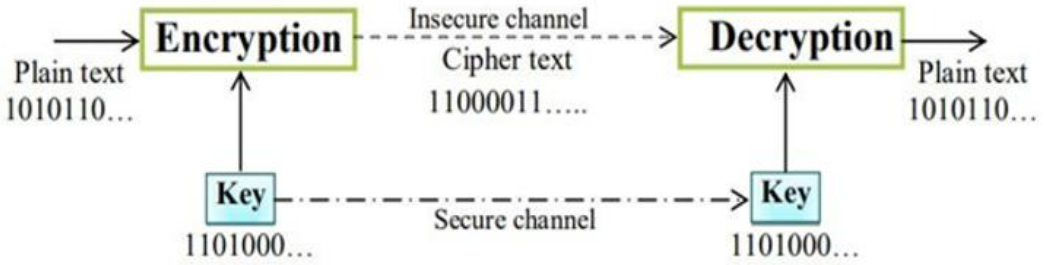


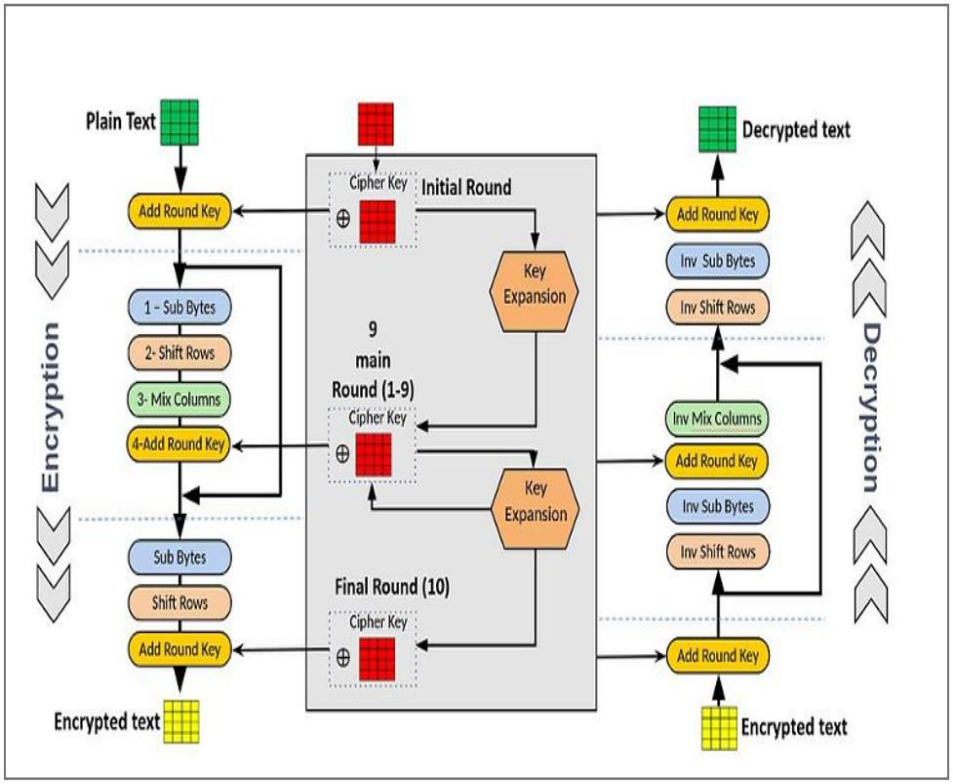
Figure 2-2 Symmetric Key Cryptography.

3.4 FLOW DIAGRAM

ADVANCED ENCRYPTION ALGORITHM

AES (Advanced Encryption Standard) is a **widely used encryption algorithm** that protects sensitive data. It turns **plain text** (readable data) into **cipher text** (unreadable code) using a **secret key**, making sure only the right person can read it by decrypting it with the same or correct key.

- AES is a symmetric key algorithm, which means the same key is used for encryption and decryption.
- It is fast, secure, and trusted by governments and big companies.
- It works on blocks of data usually 128 bits at a time.
- It can use keys of size 128, 192, or 256 bits


Plain Text
Add Round Key
1 - Sub Bytes
2- Shift Rows
3- Mix Columns
4-Add Round Key
Sub Bytes
Shift Rows
Add Round Key
Encrypted text
Encryption
Cipher Key
Initial Round
Key Expansion
9 main Round (1-9)
Cipher Key
Key Expansion
Final Round (10)
Cipher Key
Decrypted text
Add Round Key
Inv Sub Bytes
Inv Shift Rows
Inv Mix Columns
Add Round Key
Inv Sub Bytes
Inv Shift Rows
Add Round Key
Encrypted text
Decryption

3.5 AES METHODOLOGY

3.5.1 Add Round Key

The Add Round Key transformation is one of the fundamental operations in the Advanced Encryption Standard. It is the stage where the cipher directly incorporates the secret key into the data. This step is extremely important because it is the moment when the security of AES actually begins. The operation is simple but very effective due to the mathematical properties of the exclusive OR operation.

AES works on a block of data called the State. The State is one hundred and twenty eight bits in size and is usually represented as a matrix of sixteen bytes arranged in four rows and four columns. During Add Round Key, each byte of this State is combined with a byte from the Round Key using the bitwise exclusive OR operation.

The Round Key is also one hundred and twenty eight bits long in the case of AES one hundred and twenty eight. It is generated from the main secret key using the key expansion process. At this stage, both the State and the Round Key have the same size and structure, which allows each byte of the State to be directly matched with the corresponding byte of the Round Key. The Add Round Key transformation mixes the plaintext with the secret key by performing a bitwise exclusive OR of the State with the Round Key. Every round of AES contains this step and it is the only transformation that directly brings the secrecy of the key into the encryption process.

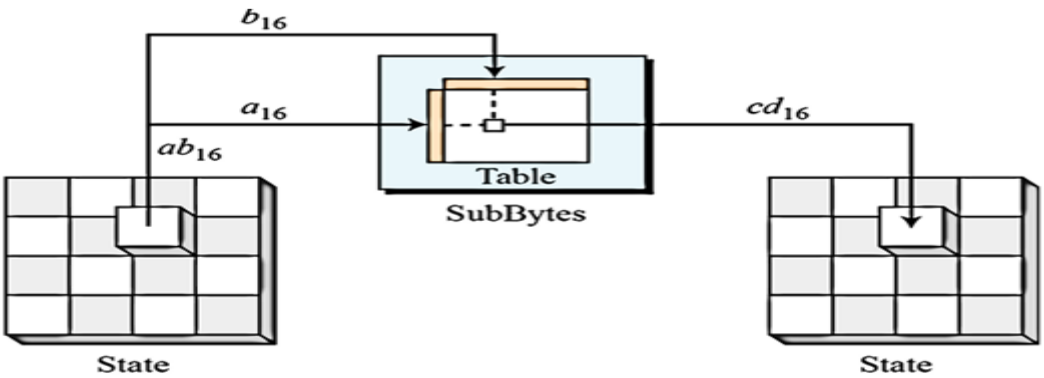
3.5.2 Substitute Bytes (S-Box)

The Substitute Bytes transformation is the stage in the Advanced Encryption Standard where each byte in the State is replaced with another value taken from a predefined substitution table known as the S Box. This process is applied independently to every byte. The purpose is to introduce a high level of nonlinearity into the encryption process.

The S Box is not a random table. It is constructed using mathematical operations in the finite field that AES is based on. Because of this design, the S Box provides strong resistance against common cryptanalytic attacks. It prevents attackers from finding predictable relationships between the input bytes and the output bytes. In other words, it removes direct patterns and ensures that small changes in the input produce large and unpredictable changes in the encrypted data.

During this transformation each byte of the State is used as an index to look up a new value in the S Box. The result replaces the original byte. Since this operation is performed for all sixteen bytes in the State, the entire block becomes more complex and difficult to analyze. This effect is commonly described as adding confusion to the cipher.

The Substitute Bytes stage uses a carefully designed S Box to replace every byte in the State with a new value. This step is essential for hiding patterns and strengthening the security of the Advanced Encryption Standard.



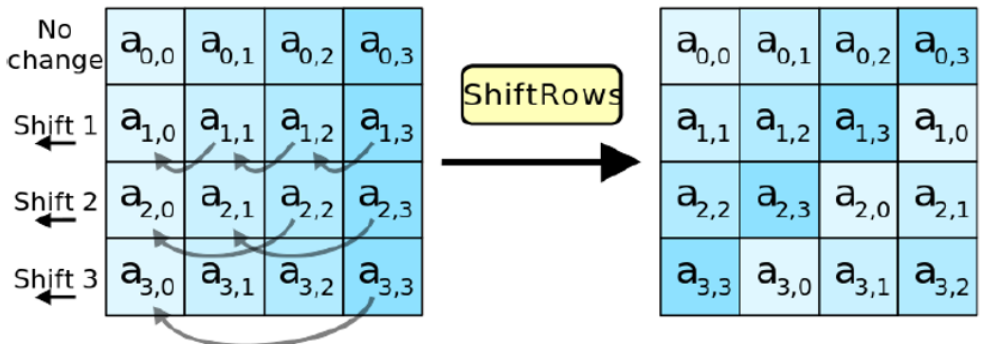
3.5.3 Shift Rows

The Shift Rows transformation is another important step in the Advanced Encryption Standard. In this stage the rows of the State matrix are shifted to the left by fixed amounts. The operation is simple, but it plays a major role in spreading information across the block.

The State in AES is arranged as a matrix with four rows and four columns. During Shift Rows, the first row is left unchanged. The second row is shifted one position to the left in a circular manner, meaning that the byte that moves past the left edge reappears at the right side of the same row. The third row is shifted two positions to the left, and the fourth row is shifted three positions to the left in the same circular style.

This transformation does not alter the actual byte values. Instead, it rearranges their positions in a controlled pattern. When combined with the other AES steps, this rearrangement increases the spread of the plaintext information across the State. Even a small change in one input byte will influence many output bytes after several rounds. This effect is known as diffusion.

The Shift Rows transformation rotates each row of the State by a predetermined amount. Although the operation is straightforward, it greatly improves the mixing of data within the block and strengthens the overall security of the Advanced Encryption Standard.

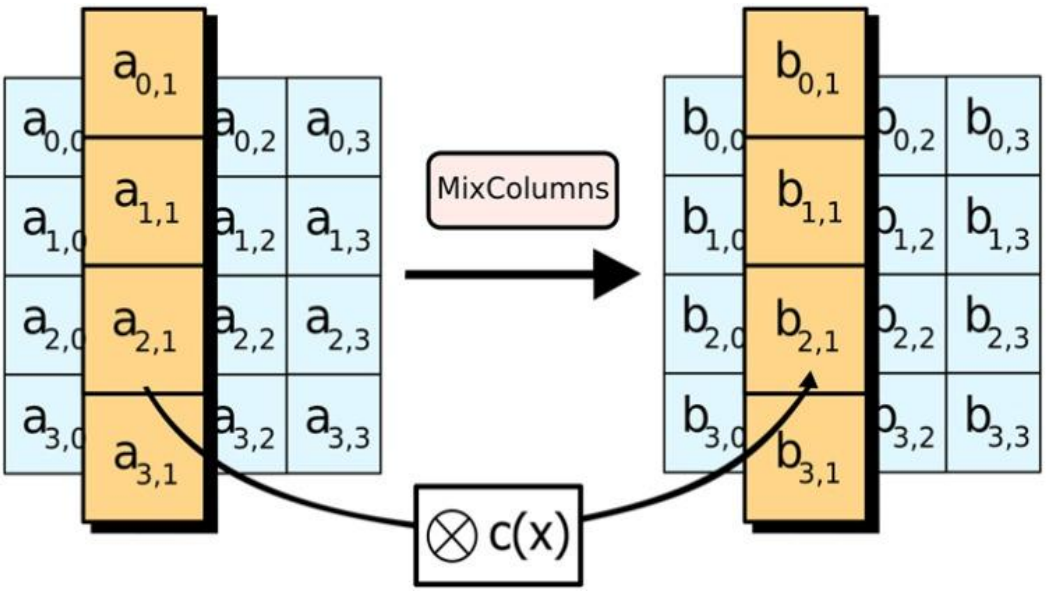


3.5.4 Mixing Columns

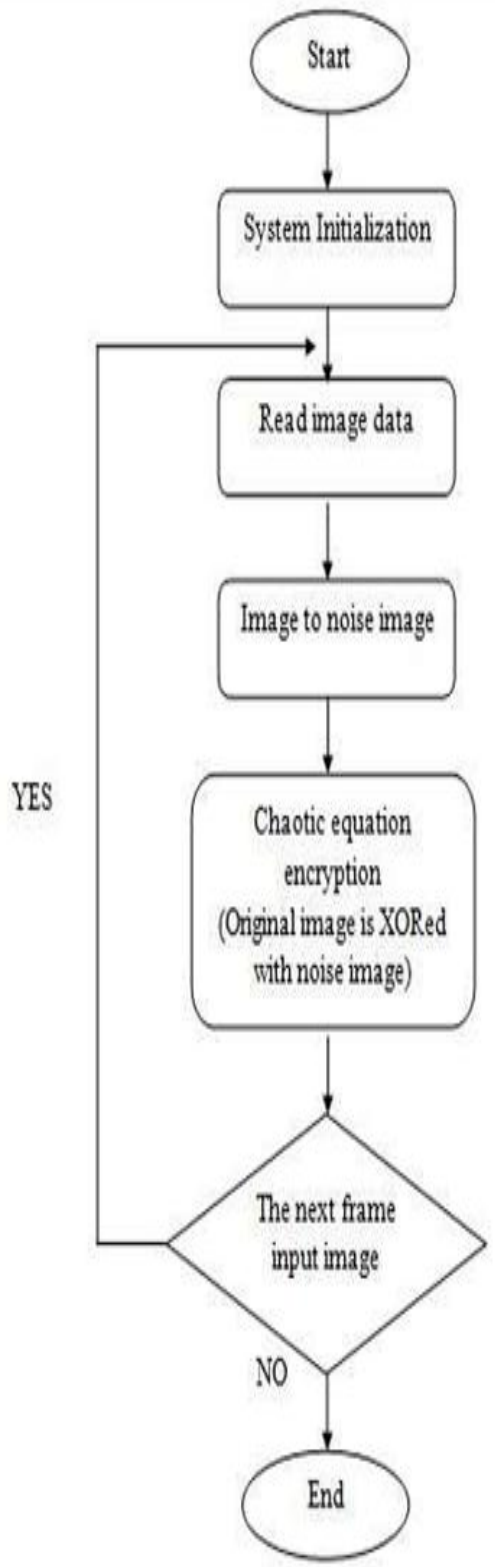
The Mix Columns transformation is the stage in the Advanced Encryption Standard where each column of the State matrix is transformed through a specific mathematical operation. Instead of changing bytes one by one, this step treats every column as a four byte vector and mixes the values together. This is done using matrix multiplication in a special algebraic structure known as the Galois Field of two raised to the power eight.

In this field, addition and multiplication follow rules that are different from normal arithmetic. These rules are chosen to ensure that the transformation produces strong diffusion. Each output byte in a column becomes a combination of all four input bytes. As a result, a small change in any one byte spreads throughout the entire column during this step.

The transformation uses a fixed four by four matrix defined by the AES standard. Each column of the State is multiplied by this matrix to produce a new column. This controlled mixing ensures that the influence of every input byte reaches far beyond its original position as the rounds progress. Because of this extensive spreading effect, Mix Columns plays a major role in making the output unpredictable, even when only one byte of the input has been altered.

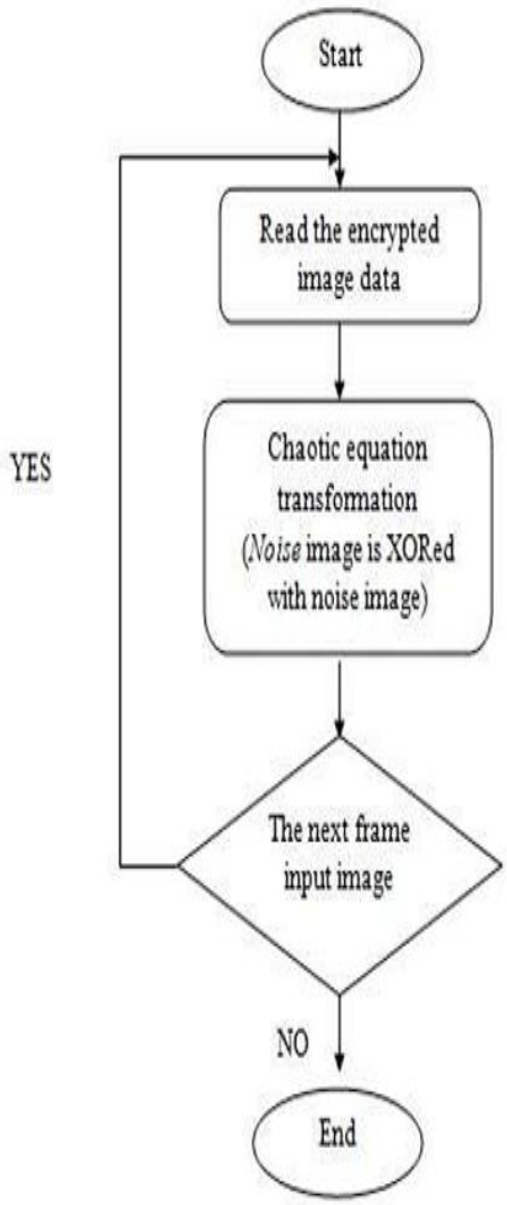


3.5.5 Encryption Process



The flowchart illustrates a frame-by-frame image encryption process using chaotic systems. The process starts with system initialization, where necessary parameters and resources are set up. It then reads the image data, which can be a single image or one frame of a video. This image is transformed into a noise image using a chaotic system, which generates pseudo-random values based on mathematical chaos equations. The original image is then encrypted by performing an XOR operation with the noise image, producing an encrypted output. The system checks if there is another image frame to process; if yes, it loops back to read and encrypt the next frame. This cycle continues until all frames are encrypted, at which point the process ends. This method ensures secure image encryption by combining the unpredictability of chaotic systems with the simplicity of XOR operations.

3.5.6 DECRYPTION PROCESS



The flowchart represents the image decryption process that corresponds to the encryption method based on chaotic systems. The process begins with the system reading the encrypted image data. It then applies a chaotic equation transformation, where the encrypted image is XORed with the same noise image used during encryption. This operation reverses the encryption, effectively retrieving the original image data due to the reversible nature of the XOR operation. After decryption, the system checks whether there is another frame to process. If there is, it loops back to read and decrypt the next frame. This cycle continues until all image frames have been decrypted, after which the process ends. This method ensures that the decryption is synchronized with the encryption process, using the same chaotic noise pattern to accurately reconstruct the original image.

CHAPTER 4
RESULTS & DISCUSSIONS

4.1 Design Overview

Our project takes an image, breaks it into data blocks, encrypts it using AES, and then decrypts it back. The system has three simple parts. The three steps are mentioned below.

4.1.1 Python part

- **Reads the image**

The Python script first loads the input image from the computer. At this stage, the image is treated as a collection of pixels with specific color values.

- **Converts the image pixels into data blocks**

Since AES works on 128-bit (16-byte) blocks, Python extracts the pixel data and groups it into chunks of 16 bytes. If the number of pixels does not fit exactly, Python adds padding to complete the last block.

- **Sends those blocks to the Verilog AES module**

Each 128-bit block is transferred from Python to the Verilog module. This communication is usually done through a file, serial interface, or co-simulation link.

- **Collects the encrypted blocks and forms the encrypted image**

After the Verilog module finishes encryption, it sends back the encrypted blocks. Python receives these blocks and arranges them back into their original positions to form the encrypted output image.

- **Does the same in reverse for decryption**

For decryption, Python repeats the same steps. It splits the encrypted image into blocks, sends them to the Verilog module for decryption, and then reassembles the decrypted data to recover the original image.

4.1.2 Verilog AES Module

❖ **This is the main hardware block**

The Verilog module acts as the core hardware engine that actually performs the AES encryption and decryption.

❖ **It performs AES encryption and decryption**

The module is designed to work in both modes. Depending on the control signal, it either encrypts the input data or decrypts it.

❖ **It takes a 128-bit block from Python**

Verilog receives one complete 128-bit block of data at a time. This block represents the pixel information grouped by Python.

❖ **Applies AES steps (Sub Bytes, Shift Rows, Mix Columns, Add Round Key)**

Inside the module, all the standard AES transformations are performed round by round.

1. **Sub Bytes**: Byte-wise substitution using S-Box
2. **Shift Rows**: Row-wise shifting to mix data
3. **Mix Columns**: Column-wise mixing for diffusion
4. **Add Round Key**: XOR operation with the round key

These steps are repeated for all AES rounds to complete encryption or decryption.

❖ **Returns encrypted or decrypted data**

- After completing all the AES rounds, the Verilog module produces the final result of the operation. This result is a 128-bit block that contains either the encrypted or decrypted form of the original data block.
- Inside the module, every AES step (Sub Bytes, Shift Rows, Mix Columns, and Add Round Key) transforms the input block round by round. Once the final round is done, the hardware stores the output in a dedicated register or output port.
- This 128-bit value is then made available on the module's output interface so that Python can read it back. Python continuously checks or waits for the hardware to signal that the block is ready. Once available, Python collects the block, stores it, and then moves on to process the next block.
- This hardware-to-software transfer continues until every block of the image has been encrypted or decrypted.

4.1.3 Working Together – Python + Verilog

1. Python prepares the data

Python acts as the main controller for the entire system. It begins by loading the input image and converting the pixel values into a one-dimensional byte stream. Since AES can only process 128-bit (16-byte) blocks, Python divides this stream into fixed-size blocks. If the final block is incomplete, Python adds padding to maintain the required block size.
Once all blocks are ready, Python sends each 128-bit block, one by one, to the Verilog AES module through the chosen communication interface. Python also sends control information such as whether the operation is encryption or decryption.

2. Verilog encrypts or decrypts the data

The Verilog module is the hardware accelerator responsible for performing the AES algorithm. After receiving a 128-bit block from Python, it applies all the steps of AES internally—Sub Bytes, Shift Rows, Mix Columns, and Add Round Key—for all required rounds.
Because hardware operates in parallel and is optimized for logical operations, the AES transformations are executed much faster compared to a software-based approach.
Once the module completes processing the block, it raises a "data ready" signal or updates the output port to indicate that the encrypted or decrypted block is available for Python to collect

3. Python rebuilds the final image

Python reads back each processed 128-bit block from the Verilog module and stores it in the correct sequence. After all blocks are retrieved, Python merges them together to form a continuous byte stream again. This byte stream is then reshaped back into the original image dimensions (height, width, color channels).
If the operation was encryption, Python outputs the image where all pixel values are now encrypted.
If the operation was decryption, Python reconstructs the original image from the decrypted blocks, restoring it to its original appearance before encryption.

3. Seamless integration (Optional point to add)

This combined approach shows how software (Python) and hardware (Verilog) work together—Python manages the image and data flow, while Verilog performs the heavy cryptographic processing. This leads to a fast, reliable, and efficient image encryption and decryption system.

4.2 Original Image (Before encrypting image)

The image displayed in this section serves as the original, unencrypted visual data. This image is used as the input to the AES encryption module implemented in Verilog, enabling comparison between the plaintext image and the encrypted output.

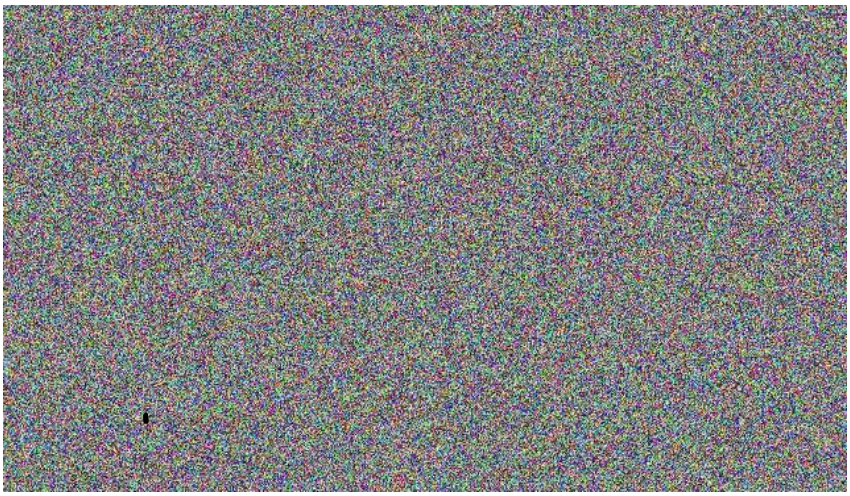
4.3 Encryption Result

When the original image was passed through the Advanced Encryption Standard encryption module, the resulting output image appeared completely scrambled and visually meaningless. This transformation occurs because AES does not work on images in a visual sense. Instead, it treats the image as raw binary data made up of pixel values. These pixel values are arranged in blocks of one hundred and twenty eight bits, and each block is processed through several rounds of AES operations, including Substitute Bytes, Shift Rows, Mix Columns, and Add Round Key.

During encryption, every byte of the pixel data is substituted, rearranged, mixed with other bytes, and combined with round keys derived from the original secret key. These steps drastically alter the numerical values that represent each pixel in the image. As a result, the spatial structure, color distribution, and identifiable patterns that existed in the original image are destroyed. The output image no longer reflects any recognizable shapes or colors from the original input.

This scrambled appearance is a strong indicator that the encryption process is functioning correctly. A secure cipher must ensure that even the slightest change in the input image or the key produces a completely different encrypted output. In the case of AES, this property is known as high diffusion and confusion. It ensures that an attacker cannot infer any meaningful information from the encrypted image, even if they have access to both the encrypted data and the general encryption procedure.

Without the correct secret key, it is computationally infeasible to reverse the operations performed by AES and recover the original pixel values. Therefore, the scrambled output image effectively protects the confidentiality of the original content. The figure shown below demonstrates the final encrypted image, highlighting how AES conceals all visual information and transforms the input into a secure, unrecognizable format.



4.4 Decryption Result

When the encrypted image was processed through the Advanced Encryption Standard decryption module using the same secret key, the system successfully restored the original image in its complete and accurate form. This demonstrates one of the core properties of Advanced Encryption Standard, which is that every transformation applied during encryption has a corresponding inverse operation. When these inverse operations are executed in the correct order with the correct key, the original data is perfectly recovered.

During decryption, the encrypted pixel data is again divided into one hundred and twenty eight bit blocks. Each block is then processed through the inverse forms of the encryption operations. The inverse Substitute Bytes reverses the substitution performed by the S Box. The inverse Shift Rows restores the original row positions by shifting them back to the right by the appropriate number of positions. The inverse Mix Columns undoes the column mixing using a specially designed inverse matrix. Finally, the Add Round Key operation is reversed by applying the same exclusive OR with the round key, which restores the original values for that round.

The round keys used during encryption are applied in the reverse order during decryption. This ensures that each step precisely cancels out the effect of the corresponding encryption step. Because the Advanced Encryption Standard algorithm is mathematically reversible, each block eventually returns to the exact set of byte values that represented the original pixels.

4.5 Waveforms

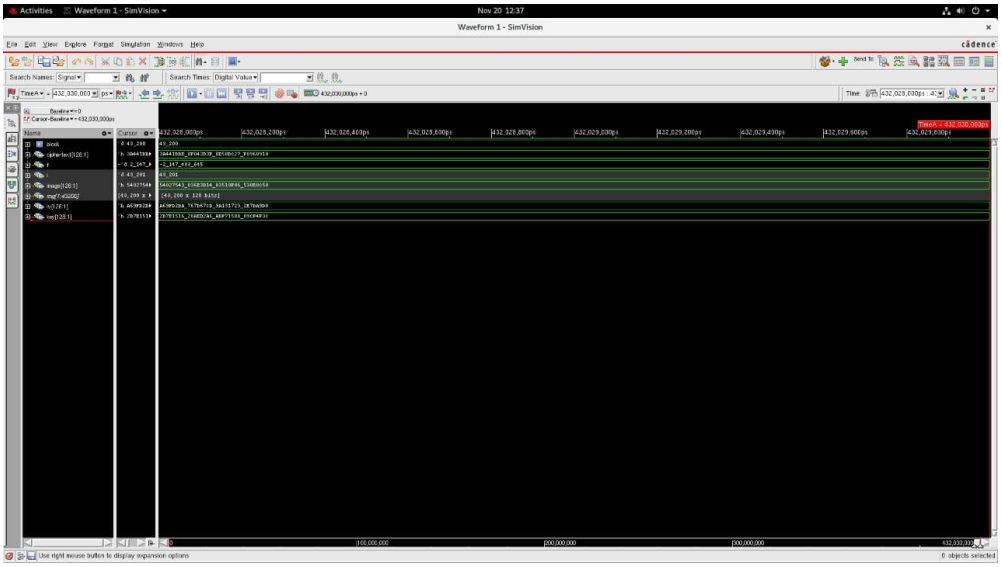


Fig. Encrypted Waveform

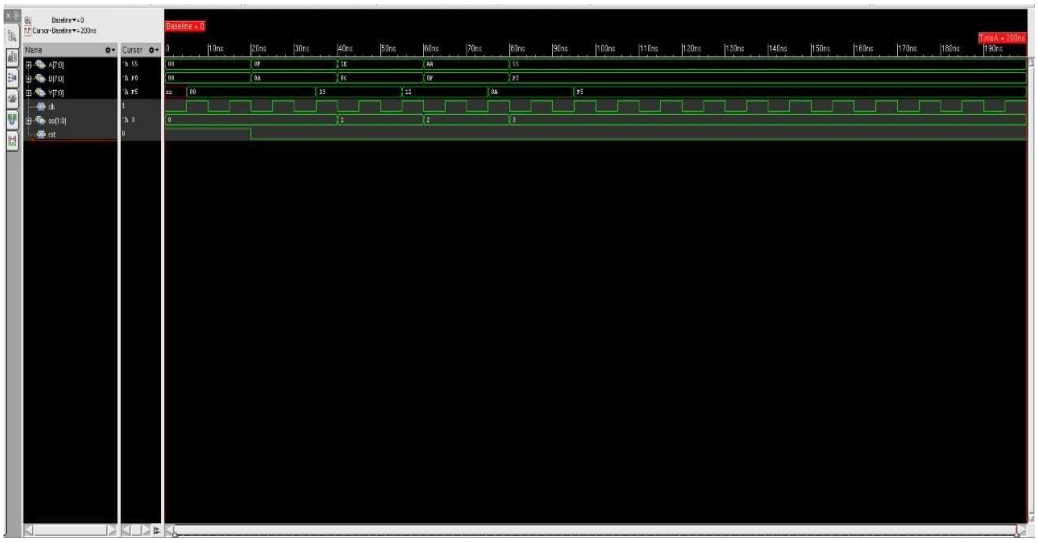


Fig. Decrypted Waveform

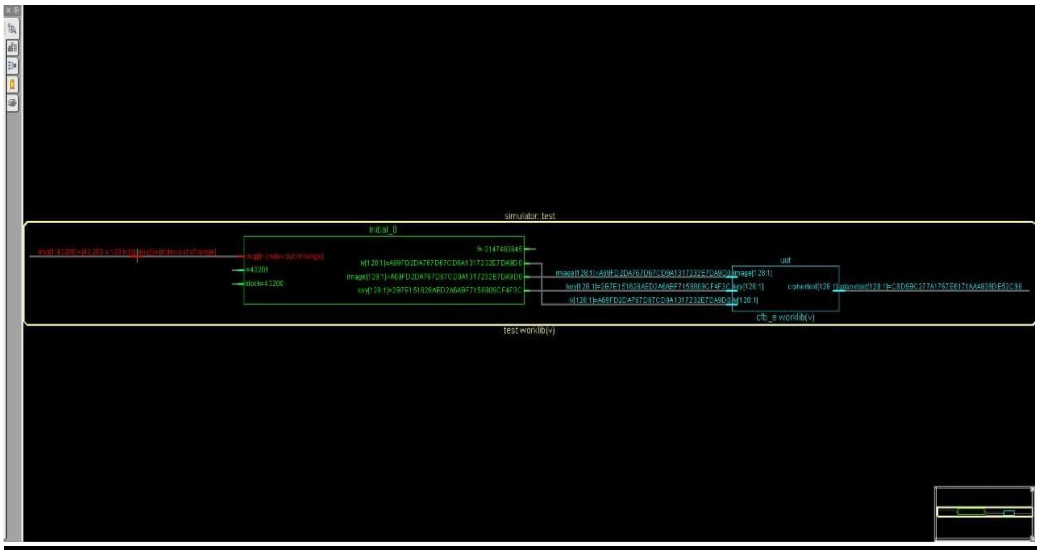


Fig. Schematic View of Encryption

CHAPTER 5
Conclusion

Summary of the System Implementation

This project successfully presents a complete image encryption and decryption system built on the Advanced Encryption Standard algorithm. The work demonstrates the collaboration of two major components. The Python component handles image preprocessing and reconstruction, while the Verilog component performs core encryption and decryption operations at the hardware level. Together these elements enable the conversion of an input image into a secure encrypted form and allow the original image to be recovered with complete accuracy when the correct secret key is applied. The approach highlights AES as a strong and dependable standard for securing digital images.

Effectiveness of AES on Image Data

A significant contribution of the project is the demonstration of how AES interacts with image data when the image is treated as structured pixel values rather than visual information. Python divides the pixel data into one hundred and twenty eight bit blocks, which are then processed in the Verilog module. Inside the hardware module all AES transformations take place such as Substitute Bytes, Shift Rows, Mix Columns, and Add Round Key.

After processing, the encrypted image becomes completely unrecognizable. No structure, color distribution, or pattern from the original image survives the encryption. This effect confirms the strength of AES and validates the expected behavior where encrypted output appears fully scrambled and meaningless to any unauthorized viewer

Correctness of Decryption

The decryption process confirms the reliability of the system. When the encrypted data enters the decryption module, the inverse AES operations restore the original pixel values with perfect accuracy. The round keys used during encryption are applied in reverse order, which cancels out every transformation. Python then reconstructs the decrypted blocks into an image that matches the original exactly. The matching output proves that the implementation of AES in hardware is correct and that all stages operate without loss or distortion.

The waveforms included in the results section support this conclusion by showing proper signal flow and completion of each AES stage

Strength of Hardware and Software Co Design

The combined use of Python and Verilog demonstrates the advantages of a mixed design environment. Python offers easy manipulation of pixel data and smooth image handling, while Verilog delivers fast performance and secure processing through parallel hardware operations. This co design structure is suitable for real world systems that require both security and efficiency such as secure communication systems, medical image protection, and confidential surveillance data handling. The project therefore reflects a practical and scalable method for securing visual information.

Security Performance of AES

The experimental results show that AES provides excellent resistance against unauthorized access due to its strong confusion and diffusion effects. Even the slightest change in the key or the input image results in completely different encrypted output. The cipher image does not reveal any useful information and cannot be interpreted in any way. This confirms the strength of AES as a security option for sensitive images and ensures that the confidentiality of the data remains protected under all circumstances.

Overall Conclusion and Future Potential

This project meets all of its objectives and demonstrates a comprehensive AES based encryption and decryption system for images. It proves that AES can be effectively implemented in hardware and can work together with software to produce a reliable security solution. The work also creates a strong groundwork for future development. Possible enhancements include real time video encryption, FPGA based deployment, higher performance through pipelining, reduced resource use, and integration with embedded systems. Overall, the project confirms that AES remains an essential and powerful standard for digital security and that a combined software and hardware approach provides a dependable method for protecting sensitive visual data.

Reference

1. V. Kachhatiya, T. S. H. Vardhan, K. Advaith and Y. R. Reddy, "**Design of Advanced Encryption Standard using Verilog-Hardware Descriptive Language**" 2025 International Conference on Recent Advances in Electrical, Electronics, Ubiquitous Communication, and Computational Intelligence (RAEEUCCI), pp. 1–5, 2025.

2. M. P. Leong, S. Z. M. Nazir and S. Y. Perng, "**Image Encryption Design using FPGA**" 2013 International Conference on Electrical, Electronics and System Engineering, pp. 27–32, 2013.

3. H. Pahuja Kumar and B. Singh, "**Design and analysis of the high speed AES using ancient Vedic mathematics novel approach**", 2016 5th International Conference on Wireless Networks and Embedded Systems (WECON), pp. 1-5, 2016.

4. R. Borhan, R. M. Fuad and Tengku Aziz, "**Successful implementation of AES algorithm in hardware**", 2012 IEEE International Conference on Electronics Design Systems and Applications (ICEDSA), pp. 27-32, 2012.

5. M. Gunasekaran, K. Rahul and S. Yachareni, "**Virtex 7 FPGA Implementation of256 Bit Key AES Algorithm with Key Schedule and Sub Bytes Block Optimization**", 2021 IEEE International IOT Electronics and Mechatronics Conference (IEMTRONICS), pp. 1-6, 2021.

6. V. H. Soumya, M. B. Neelagar and K. V. Kumaraswamy, "**Designing of AES Algorithm using Verilog**", 2018 4th International Conference for Convergence in Technology (I2CT), pp. 1-5, 2018.

7. J. Sunil, S. H. S, S. B. K and S. Santhameena, "**Implementation of AES Algorithm on FPGA and on software**", 2020 IEEE International Conference for Innovation in Technology, pp. 1 4, 2020

8. A. Kumar and S. Tejani, "**S-BOX Architecture**", Futuristic Trends in Network and Communication Technologies. FTNCT 2018. Communications in Computer and Information Science, vol. 958, 2019.

9. P. Katkade and G. M. Phade, **"Application of AES algorithm for data security in serial communication**", 2016 International Conference on Inventive Computation Technologies (ICICT), pp. 1-5, 2016.

10. M. R. Rao and R. K. Sharma, "**FPGA implementation of combined S-Box and InvS-Box of AES**", 2017 4th International Conference on Signal Processing and Integrated Networks (SPIN), pp. 566-571, 2017.